

Computer Science and Engineering Department, SVNIT, Surat
M.Tech. I – Semester I
Design and Analysis of Algorithm (CSDS103)
Lab Assignment 1

Problem Statement: 1

Problem – Employee Directory

Algorithm 1 CustomCompare

```
1: ALGORITHM CustomCompare(s1, s2):
2:   len1  $\leftarrow$  Length(s1)
3:   len2  $\leftarrow$  Length(s2)
4:   minLen  $\leftarrow$  Min(len1, len2)
5:   // Primary Tier: Case-insensitive alphabetical comparison
6:   for i  $\leftarrow$  0 TO minLen – 1 do
7:     c1  $\leftarrow$  ToLower(s1[i])
8:     c2  $\leftarrow$  ToLower(s2[i])
9:     if ASCII(c1) < ASCII(c2) then
10:      return –1
11:    else if ASCII(c1) > ASCII(c2) then
12:      return 1
13:    end if
14:  end for
15:  // Prefix Tier: Shorter prefix precedes longer string
16:  if len1 < len2 then
17:    return –1
18:  else if len1 > len2 then
19:    return 1
20:  end if
21:  // Secondary Tier: Deterministic case tie-breaking (lowercase precedes uppercase)
22:  for i  $\leftarrow$  0 TO minLen – 1 do
23:    if s1[i]  $\neq$  s2[i] then
24:      if IsLower(s1[i]) AND IsUpper(s2[i]) then
25:        return –1
26:      else if IsUpper(s1[i]) AND IsLower(s2[i]) then
27:        return 1
28:      else if ASCII(s1[i]) < ASCII(s2[i]) then
29:        return –1
30:      else
31:        return 1
32:      end if
33:    end if
34:  end for
35:  return 0
```

Algorithm 2 FindDuplicates:

```
1: ALGORITHM FindDuplicates(sortedArr):
2: duplicates  $\leftarrow$  empty list
3: n  $\leftarrow$  Length(sortedArr)
4: i  $\leftarrow$  0
5: while i < n do
6:   count  $\leftarrow$  1
7:   while i + 1 < n AND CustomCompare(sortedArr[i], sortedArr[i + 1]) = 0 do
8:     count  $\leftarrow$  count + 1
9:     i  $\leftarrow$  i + 1
10:  end while
11:  if count > 1 then
12:    Append (sortedArr[i], count) to duplicates
13:  end if
14:  i  $\leftarrow$  i + 1
15: end while
16: return duplicates
```

Code:

```
1 def compare_strings(s1, s2):
2     len1, len2 = len(s1), len(s2)
3     min_len = min(len1, len2)
4
5     for i in range(min_len):
6         c1, c2 = ord(s1[i].lower()), ord(s2[i].lower())
7         if c1 < c2:
8             return -1
9         elif c1 > c2:
10            return 1
11
12    if len1 < len2:
13        return -1
14    elif len1 > len2:
15        return 1
16
17    for i in range(min_len):
18        if s1[i] != s2[i]:
19            if s1[i].islower() and s2[i].isupper():
20                return -1
21            elif s1[i].isupper() and s2[i].islower():
22                return 1
23            elif ord(s1[i]) < ord(s2[i]):
24                return -1
25            else:
26                return 1
27
28    return 0
29
```

```
30
31 def merge(left, right):
32     result = []
33     i = j = 0
34
35     while i < len(left) and j < len(right):
36         if compare_strings(left[i], right[j]) <= 0:
37             result.append(left[i])
38             i += 1
39         else:
40             result.append(right[j])
41             j += 1
42
43     result.extend(left[i:])
44     result.extend(right[j:])
45     return result
46
47
48 def merge_sort(arr):
49     if len(arr) <= 1:
50         return arr
51
52     mid = len(arr) // 2
53     left = merge_sort(arr[:mid])
54     right = merge_sort(arr[mid:])
55
56     return merge(left, right)
57
58
59 def find_duplicates(sorted_arr):
60     duplicates = []
61     n = len(sorted_arr)
62     i = 0
63
64     while i < n:
65         count = 1
66
67         while (i + 1 < n and
68               compare_strings(sorted_arr[i],
69                               sorted_arr[i + 1]) == 0):
70             count += 1
71             i += 1
72
73         if count > 1:
74             duplicates.append((sorted_arr[i], count))
75
76         i += 1
77
78     return duplicates
79
```

```
80
81 def main():
82     raw_input = input("Enter employee names: ")
83     names = [name.strip() for name in raw_input.split(",")
84              if name.strip()]
85
86     if not names:
87         print("No valid employee names entered.")
88         return
89
90     sorted_names = merge_sort(names)
91     duplicates = find_duplicates(sorted_names)
92
93     print(f"Sorted Names: {' , '.join(sorted_names)}")
94
95     if duplicates:
96         print("Duplicate Names:")
97
98         for name, count in duplicates:
99             print(f"'{name}' appears {count} times")
100     else:
101         print("No duplicate names exist.")
102
103
104 if __name__ == "__main__":
105     main()
```

Output:

```
Enter employee names: amit, Amit, Rahul, rahul, Neha, Neha
Sorted Names: amit, Amit, Neha, Neha, rahul, Rahul
Duplicate Names:
'Neha' appears 2 times
```

Analysis:

The program successfully sorts the employee names using a custom case-insensitive lexicographical comparison while using lowercase-before-uppercase as the tie-breaking rule. For the given input of 6 employees, the sorted order is amit, Amit, Neha, Neha, rahul, Rahul. The duplicate detection step identifies Neha as the only distinct duplicate, occurring twice. The overall time complexity of the merge sort is $O(n \log n)$, while duplicate detection after sorting takes $O(n)$ time.

Problem Statement: 2

Problem – Locally Balanced Matrix

Algorithm 3 IsLocallyBalanced

```
1: ALGORITHM IsLocallyBalanced( $A, n$ ):
2: // Matrices of size  $n < 3$  have no interior elements where all 4 neighbours exist
3: if  $n < 3$  then
4:     return ( $TRUE, NULL, NULL, NULL$ )
5: end if
6: // Check all non-boundary elements ( $1 \leq i \leq n - 2, 1 \leq j \leq n - 2$ )
7: for  $i \leftarrow 1$  TO  $n - 2$  do
8:     for  $j \leftarrow 1$  TO  $n - 2$  do
9:          $neighbourSum \leftarrow A[i - 1][j] + A[i + 1][j] + A[i][j - 1] + A[i][j + 1]$ 
10:        // Check balance condition
11:        if  $A[i][j] \neq neighbourSum$  then
12:            return ( $FALSE, (i, j), A[i][j], neighbourSum$ )
13:        end if
14:    end for
15: end for
16: return ( $TRUE, NULL, NULL, NULL$ )
```

Code:

```
1 def is_locally_balanced(matrix):
2     n = len(matrix)
3     if n < 3:
4         return True, None, None, None
5
6     for i in range(1, n - 1):
7         for j in range(1, n - 1):
8             neighbour_sum = (
9                 matrix[i - 1][j] + matrix[i + 1][j]
10                + matrix[i][j - 1] + matrix[i][j + 1]
11            )
12
13            if matrix[i][j] != neighbour_sum:
14                return False, (i, j), matrix[i][j], neighbour_sum
15
16     return True, None, None, None
17
18
19 def main():
20     try:
21         n = int(input("Enter matrix size n: ").strip())
22         print(f"Enter {n} rows:")
23         matrix = []
24
25         for _ in range(n):
26             matrix.append(
```

```
27         [int(x) for x in input().replace(",", " ").split()]
28     )
29
30     balanced, pos, val, n_sum = is_locally_balanced(matrix)
31
32     if balanced:
33         print("Matrix is locally balanced.")
34     else:
35         r, c = pos
36         print("Matrix is NOT locally balanced.")
37         print(
38             f"First violation at row {r + 1}, column {c + 1}: "
39             f"Element = {val}, Neighbour Sum = {n_sum}"
40         )
41
42     except (ValueError, IndexError) as e:
43         print(f"Invalid input: {e}")
44
45
46 if __name__ == "__main__":
47     main()
```

Output:

```
Enter matrix size n: 3
Enter 3 rows:
1 2 3
5 15 1
2 7 9
Matrix is locally balanced.
```

Analysis:

The implemented algorithm correctly determines whether a matrix is locally balanced by examining only its interior elements. Boundary elements are ignored because they do not have all four required orthogonal neighbours. For each interior element, the algorithm calculates the sum of its up, down, left, and right neighbours and compares this sum with the element itself.

The algorithm traverses the matrix in row-major order and immediately stops when it encounters the first violation, reporting its position, value, and neighbour sum. This makes the approach efficient, with a worst-case time complexity of $O(n^2)$ and an auxiliary space complexity of $O(1)$. For matrices with $n < 3$, there are no interior elements, so the matrix is considered locally balanced.

The test cases demonstrate that the algorithm correctly identifies both balanced and unbalanced matrices. Overall, the solution is simple, efficient, and directly implements the given local-balance condition.

Problem Statement: 3Problem – First Unique Employee

Algorithm 4 FindFirstUniqueBruteForce

```
1: ALGORITHM FindFirstUniqueBruteForce(names):
2: n ← Length(names)
3: for i ← 0 TO n – 1 do
4:   isUnique ← TRUE
5:   for j ← 0 TO n – 1 do
6:     if i ≠ j AND names[i] = names[j] then
7:       isUnique ← FALSE
8:       BREAK
9:     end if
10:  end for
11:  if isUnique then
12:    return names[i]
13:  end if
14: end for
15: return NULL
```

Algorithm 5 FindFirstUniqueHashMap

```
1: ALGORITHM FindFirstUniqueHashMap(names):
2: freq ← Empty Hash Table
3: // Pass 1: Build frequency map
4: for all name IN names do
5:   if name ∉ freq then
6:     freq[name] ← 0
7:   end if
8:   freq[name] ← freq[name] + 1
9: end for
10: // Pass 2: Find first unique in original order
11: for all name IN names do
12:   if freq[name] = 1 then
13:     return name
14:   end if
15: end for
16: return NULL
```

Code:

```
1 def find_first_unique_brute_force(names):
2     n = len(names)
3     for i in range(n):
4         is_unique = True
5         for j in range(n):
6             if i != j and names[i] == names[j]:
7                 is_unique = False
```

```
8         break
9     if is_unique:
10         return names[i]
11 return None
12
13
14 def find_first_unique_hash_map(names):
15     freq = {}
16     for name in names:
17         freq[name] = freq.get(name, 0) + 1
18
19     for name in names:
20         if freq[name] == 1:
21             return name
22
23     return None
24
25
26 def parse_input(raw_input):
27     if not raw_input:
28         return []
29     return [name.strip() for name in raw_input.split(",") if name.strip()
30 ]
31
32 def main():
33     raw_input = input("Enter employee access sequence: ")
34     names = parse_input(raw_input)
35
36     if not names:
37         print("No valid names entered.")
38         return
39
40     unique_emp = find_first_unique_hash_map(names)
41     if unique_emp is not None:
42         print(f"First Unique Employee: {unique_emp}")
43     else:
44         print("No unique employee found.")
45
46
47 if __name__ == "__main__":
48     main()
```

Output:

```
Enter employee access sequence: kishan, kishan, rahul, neha, rahul,
shubham
First Unique Employee: neha
```

Analysis:

The problem is solved using two different approaches to identify the first employee whose name occurs exactly once in the access sequence. The brute-force approach performs repeated pairwise comparisons and requires no additional data structure, resulting in $O(1)$ auxiliary space but $O(n^2 \cdot L)$ time in the worst case. In contrast, the hash-map approach first calculates the frequency of each employee and then scans the original sequence to find the first name with frequency one.

The hash-map approach provides significantly better performance for large input sequences, with an average time complexity of $O(n \cdot L)$, at the cost of $O(u \cdot L)$ additional space, where u is the number of distinct names. Therefore, the brute-force method is suitable when memory is highly constrained or the input is small, while the hash-map approach is preferred for large-scale data due to its linear average-time performance.

The test cases confirm that both approaches correctly handle duplicate names, absence of unique employees, all-unique sequences, and cases where the unique employee appears at the end of the sequence.

Problem Statement: 4

Problem – Stable Matrix

Algorithm 6 CheckMatrixStability

```

1: ALGORITHM CheckMatrixStability( $A, n$ ):
2: // Matrices of size  $n < 3$  have no interior elements
3: if  $n < 3$  then
4:     return ( $TRUE, 0, [ ], NULL$ )
5: end if
6:  $violatingPositions \leftarrow [ ]$ 
7:  $maxDiff \leftarrow -1$ 
8:  $maxDiffInfo \leftarrow NULL$ 
9: // Traverse all non-boundary elements
10: for  $i \leftarrow 1$  TO  $n - 2$  do
11:     for  $j \leftarrow 1$  TO  $n - 2$  do
12:          $neighbourSum \leftarrow A[i - 1][j] + A[i + 1][j] + A[i][j - 1] + A[i][j + 1]$ 
13:          $diff \leftarrow Abs(A[i][j] - neighbourSum)$ 
14:         if  $diff > 0$  then
15:             Append  $(i, j)$  to  $violatingPositions$ 
16:             if  $diff > maxDiff$  then
17:                  $maxDiff \leftarrow diff$ 
18:                  $maxDiffInfo \leftarrow ((i, j), A[i][j], neighbourSum, diff)$ 
19:             end if
20:         end if
21:     end for
22: end for
23: if  $Length(violatingPositions) = 0$  then
24:     return ( $TRUE, 0, [ ], NULL$ )
25: else
26:     return ( $FALSE, Length(violatingPositions), violatingPositions, maxDiffInfo$ )
27: end if

```

Code:

```

1 def check_matrix_stability(matrix):
2     n = len(matrix)
3     if n < 3:
4         return True, 0, [], None
5
6     violating_positions = []
7     max_diff = -1
8     max_diff_info = None
9
10    for i in range(1, n - 1):
11        for j in range(1, n - 1):
12            neighbour_sum = (matrix[i - 1][j] + matrix[i + 1][j] +
13                            matrix[i][j - 1] + matrix[i][j + 1])
14            diff = abs(matrix[i][j] - neighbour_sum)
15

```

```
16         if diff > 0:
17             violating_positions.append((i, j))
18             if diff > max_diff:
19                 max_diff = diff
20                 max_diff_info = ((i, j), matrix[i][j],
21                                 neighbour_sum, diff)
22
23     if not violating_positions:
24         return True, 0, [], None
25
26     return False, len(violating_positions), \
27             violating_positions, max_diff_info
28
29
30 def main():
31     try:
32         n = int(input("Enter matrix size n: ").strip())
33         print(f"Enter {n} rows:")
34         matrix = []
35         for _ in range(n):
36             matrix.append(
37                 [int(x) for x in input().replace(",", " ").split()]
38             )
39
40         is_stable, count, positions, max_info = \
41             check_matrix_stability(matrix)
42
43         if is_stable:
44             print("Matrix is stable.")
45         else:
46             print("Matrix is NOT stable.")
47             print(f"Total violating elements: {count}")
48             print(f"Violating positions: {positions}")
49
50             if max_info:
51                 (r, c), val, n_sum, diff = max_info
52                 print(
53                     f"Maximum difference at row {r + 1}, column {c +
54 1}: "
55                     f"Element = {val}, Neighbour Sum = {n_sum}, "
56                     f"Max |Diff| = {diff}"
57                 )
58
59     except (ValueError, IndexError) as e:
60         print(f"Invalid input: {e}")
61
62 if __name__ == "__main__":
63     main()
```

Output:

```
Enter matrix size n: 3
Enter 3 rows:
1 2 3
5 15 8
4 0 5
Matrix is stable.
```

Analysis:

The stable matrix algorithm checks every interior element of the matrix and compares its value with the sum of its four orthogonal neighbours. Unlike the locally balanced matrix algorithm, this approach does not stop at the first violation. Instead, it records all violating positions and continuously maintains the maximum absolute difference between an element and its neighbour sum.

Since every interior element must be examined to determine the complete set of violations and the maximum difference, the algorithm has a time complexity of $\Theta(n^2)$ in the best, average, and worst cases. No additional matrix is required because the algorithm only reads the input matrix and maintains a few scalar variables. However, storing the violating positions requires $O(k)$ space, where k is the number of violating elements.

The test cases demonstrate that the algorithm correctly identifies stable matrices, counts all violations in unstable matrices, records their positions, and determines the violation with the maximum absolute difference. Thus, the algorithm provides a complete and memory-efficient analysis of matrix stability.

Problem Statement: 5

Problem – Nearly Ordered Employee Directory

Algorithm 7 AnalyzeNearlyOrderedDirectory

```
1: ALGORITHM AnalyzeNearlyOrderedDirectory(names):
2:  $n \leftarrow \text{Length}(\text{names})$ 
3: if  $n \leq 1$  then
4:   return (ALREADY_SORTED, [ ], NULL)
5: end if
6:  $\text{violations} \leftarrow [ ]$ 
7: // Step 1: Detect all adjacent drops in a single linear pass
8: for  $i \leftarrow 0$  TO  $n - 2$  do
9:   if  $\text{CompareStrings}(\text{names}[i], \text{names}[i + 1]) > 0$  then
10:    Append  $i$  to  $\text{violations}$ 
11:   end if
12: end for
13: if  $\text{Length}(\text{violations}) = 0$  then
14:   return (ALREADY_SORTED, [ ], NULL)
15: end if
16: // Step 2: Identify candidate swap positions
17: if  $\text{Length}(\text{violations}) = 1$  then
18:    $x \leftarrow \text{violations}[0]$ 
19:    $y \leftarrow \text{violations}[0] + 1$ 
20: else if  $\text{Length}(\text{violations}) = 2$  then
21:    $x \leftarrow \text{violations}[0]$ 
22:    $y \leftarrow \text{violations}[1] + 1$ 
23: else
24:   // More than 2 drops cannot be resolved by a single swap
25:   return (CANNOT_BE_SORTED_BY_ONE_SWAP,  $\text{violations}$ , NULL)
26: end if
27: // Step 3: Verify whether swapping names[x] and names[y] makes list sorted
28:  $\text{Swap}(\text{names}[x], \text{names}[y])$ 
29:  $\text{isSorted} \leftarrow \text{TRUE}$ 
30: for  $k \leftarrow 0$  TO  $n - 2$  do
31:   if  $\text{CompareStrings}(\text{names}[k], \text{names}[k + 1]) > 0$  then
32:      $\text{isSorted} \leftarrow \text{FALSE}$ 
33:     BREAK
34:   end if
35: end for
36:  $\text{Swap}(\text{names}[x], \text{names}[y])$  ▷ Restore original order
37: if  $\text{isSorted}$  then
38:   return (FIXABLE_BY_SWAP,  $\text{violations}$ ,  $(x, y)$ )
39: else
40:   return (CANNOT_BE_SORTED_BY_ONE_SWAP,  $\text{violations}$ , NULL)
41: end if
```

Code:

```
1 def compare_strings(s1, s2):
2     min_len = min(len(s1), len(s2))
3     for i in range(min_len):
4         c1_low, c2_low = s1[i].lower(), s2[i].lower()
5         if c1_low != c2_low:
6             return -1 if c1_low < c2_low else 1
7
8     if len(s1) != len(s2):
9         return -1 if len(s1) < len(s2) else 1
10
11    for i in range(len(s1)):
12        if s1[i] != s2[i]:
13            if s1[i].islower() and s2[i].isupper():
14                return -1
15            if s1[i].isupper() and s2[i].islower():
16                return 1
17            return -1 if s1[i] < s2[i] else 1
18
19    return 0
20
21
22 def analyze_directory_ordering(names):
23     n = len(names)
24     if n <= 1:
25         return "ALREADY_SORTED", [], None, names.copy()
26
27     violations = []
28     for i in range(n - 1):
29         if compare_strings(names[i], names[i + 1]) > 0:
30             violations.append(i)
31
32     if not violations:
33         return "ALREADY_SORTED", [], None, names.copy()
34
35     if len(violations) == 1:
36         x, y = violations[0], violations[0] + 1
37     elif len(violations) == 2:
38         x, y = violations[0], violations[1] + 1
39     else:
40         return "UNFIXABLE_BY_ONE_SWAP", violations, None, None
41
42     names_copy = names.copy()
43     names_copy[x], names_copy[y] = names_copy[y], names_copy[x]
44
45     for k in range(n - 1):
46         if compare_strings(names_copy[k], names_copy[k + 1]) > 0:
47             return "UNFIXABLE_BY_ONE_SWAP", violations, None, None
48
```

```
49     return "FIXABLE_BY_SWAP", violations, (x, y), names_copy
50
51
52 def parse_input(raw_input):
53     if not raw_input:
54         return []
55     return [name.strip() for name in raw_input.split(",") if name.strip()
56 ]
57
58 def main():
59     raw_input = input("Enter employee names: ")
60     names = parse_input(raw_input)
61
62     if not names:
63         print("No valid names entered.")
64         return
65
66     status, violations, swap_pos, fixed_list = \
67         analyze_directory_ordering(names)
68
69     if status == "ALREADY_SORTED":
70         print("Directory is already sorted.")
71     elif status == "FIXABLE_BY_SWAP":
72         x, y = swap_pos
73         print(
74             f"Directory is NOT sorted. Violations at indices: {
violations}"
75         )
76         print(
77             f"Can be sorted by swapping position {x + 1} "
78             f"('{names[x]}') and position {y + 1} ('{names[y]}')."
79         )
80         print(f"Sorted directory: {' '.join(fixed_list)}")
81     else:
82         print(
83             f"Directory is NOT sorted. Violations at indices: {
violations}"
84         )
85         print("Cannot be sorted by exchanging only two names.")
86
87
88 if __name__ == "__main__":
89     main()
```

Output:

```
Enter employee names: amit, neha, karan, rahul, priya
Directory is NOT sorted. Violations at indices: [1, 3]
Cannot be sorted by exchanging only two names.
```

Analysis:

The algorithm efficiently analyzes whether an employee directory is already in lexicographical order and, if not, determines whether the ordering can be restored using a single swap. It first performs a linear scan to identify adjacent ordering violations. Based on the number and positions of these violations, it selects candidate elements for swapping and verifies the resulting sequence without sorting the complete directory.

The algorithm requires $O(n \cdot L)$ time, where n is the number of employee names and L is the maximum name length. This is more efficient than sorting the entire directory, which would require $O(n \log n \cdot L)$ time. The auxiliary space complexity is $O(1)$ apart from the small list used to store violation indices.

A single swap can restore the ordering when there is at most two relevant adjacent violations and the selected elements satisfy the required boundary and intermediate ordering conditions. The test cases demonstrate all important scenarios, including an already sorted directory, adjacent and non-adjacent swaps, and cases that cannot be corrected using only one swap.

Problem Statement: 6

Problem – Dominant Element

Algorithm 8 FindDominantBoyerMoore

```
1: ALGORITHM FindDominantBoyerMoore( $A, n$ ):
2:   if  $n = 0$  then
3:     return NULL
4:   end if
5:   // Phase 1: Candidate Selection
6:   candidate  $\leftarrow$  NULL
7:   count  $\leftarrow$  0
8:   for all  $x$  IN  $A$  do
9:     if count = 0 then
10:      candidate  $\leftarrow$   $x$ 
11:      count  $\leftarrow$  1
12:    else if  $x = \textit{candidate}$  then
13:      count  $\leftarrow$  count + 1
14:    else
15:      count  $\leftarrow$  count - 1
16:    end if
17:  end for
18:  // Phase 2: Mandatory Verification
19:  actualCount  $\leftarrow$  0
20:  for all  $x$  IN  $A$  do
21:    if  $x = \textit{candidate}$  then
22:      actualCount  $\leftarrow$  actualCount + 1
23:    end if
24:  end for
25:  if actualCount >  $\text{Floor}(n/2)$  then
26:    return (candidate, actualCount)
27:  else
28:    return NULL
29:  end if
```

Code:

```
1 def find_dominant_brute_force(arr):
2     n = len(arr)
3     threshold = n // 2
4
5     for i in range(n):
6         count = 0
7         for j in range(n):
8             if arr[j] == arr[i]:
9                 count += 1
10        if count > threshold:
11            return arr[i], count
12
13    return None
```

```
14
15
16 def find_dominant_boyer_moore(arr):
17     n = len(arr)
18     if n == 0:
19         return None
20
21     threshold = n // 2
22     candidate = None
23     count = 0
24
25     for num in arr:
26         if count == 0:
27             candidate = num
28             count = 1
29         elif num == candidate:
30             count += 1
31         else:
32             count -= 1
33
34     if candidate is None:
35         return None
36
37     actual_count = 0
38     for num in arr:
39         if num == candidate:
40             actual_count += 1
41
42     if actual_count > threshold:
43         return candidate, actual_count
44
45     return None
46
47
48 def parse_input(raw_input):
49     if not raw_input:
50         return []
51     return [int(x) for x in raw_input.replace(",", " ").split()]
52
53
54 def main():
55     raw_input = input("Enter integers: ")
56     arr = parse_input(raw_input)
57
58     if not arr:
59         print("No valid integers entered.")
60         return
61
62     result = find_dominant_boyer_moore(arr)
63     if result:
```

```
64     elem, count = result
65     print(
66         f"Dominant Element: {elem} "
67         f"(Count: {count}, Threshold: > {len(arr) // 2})"
68     )
69 else:
70     print("No dominant element exists.")
71
72
73 if __name__ == "__main__":
74     main()
```

```
Enter integers: 1,2,3,4,4,4,4,4
Dominant Element: 4 (Count: 5, Threshold: > 4)
```

Analysis:

The problem determines whether an integer array contains a dominant element whose frequency is strictly greater than half of the array size. Two approaches are considered: repeated counting and the Boyer–Moore Majority Voting Algorithm. The brute-force approach checks the frequency of each element using nested loops, resulting in $O(n^2)$ worst-case time and $O(1)$ auxiliary space.

The Boyer–Moore approach is more efficient. It first identifies a possible majority candidate through pairwise cancellation and then performs a second pass to verify its actual frequency. It requires $\Theta(n)$ time in all cases and $O(1)$ auxiliary space, without modifying the input array. Therefore, it provides an optimal solution when both time and memory efficiency are important.

The test cases confirm that the algorithm correctly handles dominant elements, arrays without a dominant element, identical elements, single-element arrays, and negative integers. Thus, the dominant element can be identified without any additional data structure using the Boyer–Moore algorithm.